

Como a aplicação de chat funciona

Uma leitura por componentes e decisões técnicas

`Message.java`

`ChatServer.java`

`ClientHandler.java`

`ChatClient.java`

Três peças que formam o sistema



Pacote Comum

Message.java

Define o formato dos dados que trafegam pela rede. Ao ser criada, armazena remetente, texto e a hora exata da mensagem.



O Servidor

ChatServer.java
ClientHandler.java

Espinha dorsal da aplicação. Gerencia conexões TCP e UDP ao mesmo tempo, e distribui mensagens para todos os clientes.



O Cliente

ChatClient.java

Fornece a interface gráfica (Swing) e orquestra o envio de texto via TCP e o cálculo de ping via UDP.

Message.java — O formato dos dados

O que uma mensagem carrega

`sender`

Nome do remetente — quem enviou a mensagem

`text`

O conteúdo da mensagem digitada

`timestamp`

Hora capturada no momento da criação (`LocalTime.now()`)

Por que `Serializable`?

A interface `Serializable` permite que o Java converta o objeto `Message` inteiro em bytes e o envie pelo cabo de rede via `ObjectOutputStream` — sem ela, não seria possível enviar o objeto diretamente.

`toString()` — exibição no chat

O método `toString()` formata a saída que aparece na janela do chat:

```
[14:32:10] Alice: Olá pessoal!
```

ChatServer.java

Loop TCP — aceitar clientes

ServerSocket em modo de escuta

Fica aguardando na porta 12345. Quando um cliente se conecta, o método `accept()` desbloqueia e retorna o novo socket.

Uma Thread por cliente

Para cada nova conexão, o servidor cria um `ClientHandler` e o coloca em sua própria thread — assim, um cliente nunca bloqueia os outros.

`broadcast()` para todos

Ao receber uma mensagem, o servidor a repassa para TODOS os clientes da lista, incluindo quem enviou.

Thread UDP — responder pings

Roda em paralelo, em background (thread daemon). Fica ouvindo a porta 12346.

Como funciona o ping-pong UDP:

- ① Cliente envia datagrama PING
- ② Servidor recebe e devolve o mesmo pacote
- ③ Cliente mede: `tempo_pong - tempo_ping`
- ④ Resultado exibido como 'Latência UDP: X ms'

ClientHandler.java — Um gerente por cliente

1

Identificação

Lê a primeira mensagem recebida e trata seu campo "sender" como o nome do usuário. Anuncia para todos: "X entrou no chat."

2

Loop de escuta

Entra em um loop infinito aguardando mensagens. A cada mensagem recebida, chama broadcast() no servidor para repassá-la a todos.

3

Desconexão

Quando o cliente fecha a janela, ocorre EOFException (fim do stream). O bloco finally remove o cliente da lista e avisa no chat: "X saiu."

ChatClient.java — Três tarefas ao mesmo tempo

T1 Event Dispatch Thread (EDT)

Gerencia toda a interface gráfica

- ▶ Responde a cliques e teclas do usuário
- ▶ Atualiza a área de chat (JTextArea)
- ▶ NUNCA executa operações de rede — se travar, a janela congela

T2 TCP-Receiver

Recebe mensagens do servidor

- ▶ Fica bloqueada aguardando `in.readObject()`
- ▶ Ao receber, usa `invokeLater()` para atualizar a tela
- ▶ Roda como daemon: encerra com a aplicação

T3 UDP-Ping

Mede a latência da rede

- ▶ Envia PING e aguarda PONG a cada 2 segundos
- ▶ Calcula RTT e atualiza o painel de latência
- ▶ Se pacote se perde → exibe 'timeout' e continua

Por que o código foi feito assim?



TCP para o chat, UDP para o ping

TCP garante entrega ordenada — mensagens perdidas ou fora de ordem destruiriam uma conversa. UDP tem menos overhead e é tolerante a falhas: se um pacote de ping se perder, o código captura o timeout e tenta novamente em 2 segundos.



CopyOnWriteArrayList no servidor

Múltiplos ClientHandlers leem a lista de clientes para broadcast enquanto outros entram/saem (modificando-a). O CopyOnWriteArrayList cria cópias internas em cada gravação, evitando o ConcurrentModificationException — sem travar o código com synchronized.



out.reset() após cada envio

ObjectOutputStream mantém cache dos objetos enviados para economizar banda. Sem o reset(), o Java entenderia que uma nova mensagem é "o mesmo objeto de antes" e enviaria uma referência — o receptor receberia sempre a primeira mensagem repetida.

Do texto digitado à tela de todos



Em paralelo — Medição de Latência UDP:

